

./media/logo-upt.png

Figure 1: ./media/logo-upt.png

UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

Proyecto de Antivirus

Curso: *Calidad y Pruebas de Software*

Docente: *Mag. Patrick Cuadros Quiroga*

Integrantes:

LLica Mamani, Jimmy Mijair (2023076789)

Sierra Ruiz, Iker Alberto (2023077090)

Tacna – Perú

2026

Sistema *SecureGuard Antivirus*

Informe de Gestión del Proyecto

Versión *1.0*

CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha
1.0	LLica Mamani, Jimmy Mijair	Sierra Ruiz, Iker Alberto	LLica Mamani, Jimmy Mijair	01/05/2026

ÍNDICE GENERAL

1. Introducción
2. Uso de GitHub en el Proyecto
3. Estructura del Repositorio
4. GitHub Wiki
5. Roadmap del Proyecto

6. Flujo de Trabajo con Ramas (GitFlow)
7. GitHub Projects
8. Conclusiones

Informe de Gestión del Proyecto

1. Introducción

1.1. Propósito

El presente informe describe la gestión de configuración y del proyecto SecureGuard Antivirus, detallando el uso de las herramientas de GitHub para el control de versiones, la planificación del roadmap, el flujo de trabajo colaborativo y el seguimiento del progreso del proyecto.

1.2. Alcance

Este informe cubre la gestión del repositorio GitHub del proyecto SecureGuard Antivirus, incluyendo la estructura del repositorio, el flujo de trabajo con ramas, el uso de GitHub Projects para el seguimiento de tareas, el roadmap de versiones y el uso de la GitHub Wiki como centro de documentación.

1.3. Referencias

Documento	Descripción
FD01-Informe-Factibilidad.md	Análisis de viabilidad del proyecto
FD02-Informe-Vision.md	Visión y alcance del sistema
FD03-Informe-Requerimientos.md	Especificación de requerimientos
FD04-Informe-Arquitectura.md	Arquitectura del sistema

2. Uso de GitHub en el Proyecto

2.1. Rol de GitHub en SecureGuard Antivirus

GitHub es la plataforma central de gestión para el proyecto SecureGuard Antivirus. Se utiliza para:

Funcionalidad GitHub	Uso en el Proyecto
Repositorio	Almacenamiento y versionado del código fuente (Rust + Python) y documentación
Issues	Registro de requerimientos, tareas, bugs y mejoras como historias de usuario

Funcionalidad GitHub	Uso en el Proyecto
Pull Requests	Revisión de código entre integrantes antes de integrar cambios
GitHub Projects	Tablero Kanban para seguimiento del sprint y fases del proyecto
GitHub Wiki	Documentación técnica y de usuario del sistema
GitHub Actions	(Planificado v1.1) CI/CD para compilación y pruebas automáticas
Releases	Empaquetado y publicación de versiones entregables del antivirus
GitHub Pages	(Planificado v2.0) Landing page del proyecto

2.2. Convenciones de Commits

Se aplica la especificación **Conventional Commits** para mantener un historial de cambios legible y semánticamente significativo:

Tipo	Descripción	Ejemplo
<code>feat</code>	Nueva funcionalidad	<code>feat: agregar escaneo heurístico de entropía</code>
<code>fix</code>	Corrección de error	<code>fix: corregir falso positivo en detección de strings</code>
<code>docs</code>	Cambios de documentación	<code>docs: actualizar README con instrucciones de instalación</code>
<code>refactor</code>	Refactorización sin cambio funcional	<code>refactor: separar lógica de cuarentena en módulo independiente</code>
<code>test</code>	Adición o modificación de pruebas	<code>test: agregar prueba unitaria para scanner.rs</code>
<code>chore</code>	Tareas de mantenimiento	<code>chore: actualizar dependencias Cargo.toml</code>

Tipo	Descripción	Ejemplo
ci	Cambios en CI/CD	ci: configurar GitHub Actions para compilación Rust

2.3. Políticas del Repositorio

- Todo código debe pasar revisión de un integrante antes de fusionarse a `main`.
- Los commits directos a `main` están restringidos; se utilizan Pull Requests.
- Las issues se cierran automáticamente al fusionar el PR asociado mediante el keyword `Closes #N`.
- Las ramas de funcionalidad se eliminan después de ser fusionadas.

3. Estructura del Repositorio

3.1. Organización de Directorios

proyecto_antivirus/

```

codigo-fuente/                # Código fuente del proyecto
  backend_rust/                # Motor del antivirus en Rust
    src/
      main.rs                  # Punto de entrada del motor
      scanner.rs               # Motor de escaneo (hash + heurística)
      daemon.rs                # Manejador de comandos del daemon
      quarantine.rs            # Gestión de cuarentena
      network.rs               # Control del firewall de Windows
      cleaner.rs               # Limpieza de archivos temporales
      updater.rs               # Actualización de firmas
      protection.rs            # Protección en tiempo real
      utils.rs                 # Utilidades comunes
    Cargo.toml                 # Dependencias y metadata del proyecto Rust
    Cargo.lock                 # Lock de versiones de dependencias

  frontend_python/            # Interfaz gráfica en Python
    views/                     # Vistas de la interfaz
      dashboard_view.py        # Vista principal / dashboard
      protection_view.py       # Vista de módulos de protección
      firewall_view.py         # Vista de control del firewall
      cleaning_view.py         # Vista de limpieza del sistema
      quarantine_view.py       # Vista de gestión de cuarentena
      settings_view.py         # Vista de configuración / sistema
      update_view.py           # Vista de actualizaciones
      __init__.py

```

```

assets/                # Recursos gráficos (íconos, imágenes)
main.py                # Punto de entrada de la GUI
engine_bridge.py       # Puente de comunicación GUI motor Rust
utils.py               # Utilidades (rutas de recursos)
requirements.txt        # Dependencias Python

Instalador/            # Binarios compilados del instalador
  BitCraft_Antivirus/  # Ejecutable empaquetado con PyInstaller

media/                 # Recursos de documentación
  logo-upt.png          # Logo institucional

FD01-Informe-Factibilidad.md  # Informe de factibilidad
FD02-Informe-Vision.md       # Informe de visión
FD02-Informe-Gestion.md      # Informe de gestión del proyecto
FD03-Informe-Requerimientos.md # Informe de requerimientos
FD04-Informe-Arquitectura.md # Informe de arquitectura
README.md                   # Descripción principal del proyecto

```

3.2. Archivos de Configuración Importantes

Archivo	Ubicación	Propósito
Cargo.toml	backend_rust/	Dependencias y configuración del proyecto Rust
requirements.txt	frontend_python/	Dependencias Python de la GUI
.gitignore	codigo-fuente/	Exclusión de binarios y archivos temporales
BitCraft_Antivirus.spc	frontend_python/	Configuración de empaquetado PyInstaller

4. GitHub Wiki

4.1. Estructura de la Wiki

La GitHub Wiki del proyecto sirve como centro de documentación técnica y de usuario. Está organizada en las siguientes páginas principales:

Página Wiki	Contenido
Home	Bienvenida, descripción del proyecto, enlaces rápidos
Características	Listado detallado de funcionalidades del antivirus

Página Wiki	Contenido
Instalación	Guía paso a paso para instalar y ejecutar SecureGuard
Arquitectura	Descripción de la arquitectura del sistema y diagramas
Roadmap	Versiones planificadas con fechas y funcionalidades

4.2. Políticas de la Wiki

- La Wiki se actualiza simultáneamente con cada release.
- Las páginas técnicas son responsabilidad del equipo de desarrollo.
- Se utiliza Markdown estándar con diagramas Mermaid para la documentación visual.
- Cada cambio significativo en la arquitectura o API debe reflejarse en la Wiki antes de la integración a `main`.

5. Roadmap del Proyecto

5.1. Visión General del Roadmap

El roadmap de SecureGuard Antivirus está dividido en tres versiones principales, cada una con objetivos claros y funcionalidades incrementales.

5.2. Versión v1.0 — Producto Mínimo Viable (MVP)

Fecha estimada de lanzamiento: 30 de mayo de 2026

Objetivos: Entregar un antivirus funcional con las capacidades de protección esenciales.

Funcionalidad	Estado	Issue
Motor de escaneo basado en hashes SHA-256	Completado	#7
Detección heurística por entropía (archivos PE)	Completado	#7
Detección por strings sospechosos	Completado	#7
Protección en tiempo real (file watcher)	Completado	#8
Gestión de cuarentena (mover/restaurar/eliminar)	Completado	#9
Control del firewall de Windows	Completado	#10
Limpieza de archivos temporales	Completado	#11
Actualización de firmas desde repositorio	Completado	#12

Funcionalidad	Estado	Issue
Interfaz gráfica con CustomTkinter	Completado	#13
Dashboard con indicador de estado	Completado	#13
Vista de protección con switches	Completado	#13
Vista de firewall independiente	Completado	#14
Documentación académica (FD01–FD04)	Completado	#1–#4

5.3. Versión v1.1 — Mejoras de Calidad y CI/CD

Fecha estimada de lanzamiento: 31 de julio de 2026

Objetivos: Aumentar la calidad del código, automatizar pruebas y mejorar la experiencia de usuario.

Funcionalidad	Descripción
CI/CD con GitHub Actions	Pipeline automático: compilación Rust + pruebas pytest
Cobertura de pruebas unitarias 80%	Pruebas para scanner, quarantine, updater, cleaner
Pruebas de integración frontend backend	Validar protocolo JSON del EngineBridge
Notificaciones del sistema (plyer)	Alertas de escritorio al detectar amenazas
Historial de escaneos	Registro de escaneos anteriores con fecha y resultados
Anti-Ransomware mejorado	Detección de patrones de cifrado masivo de archivos
Soporte multiidioma (ES/EN)	Internacionalización de la interfaz
Actualizaciones automáticas programadas	Cron job para actualizar firmas cada 24 horas
Firma de commits GPG	Verificación de integridad del código
GitHub Security Advisories	Reporte formal de vulnerabilidades

5.4. Versión v2.0 — Plataforma de Seguridad Completa

Fecha estimada de lanzamiento: 31 de enero de 2027

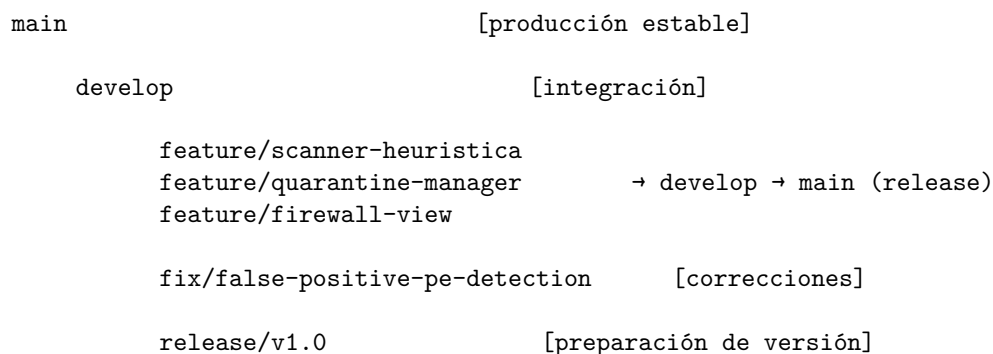
Objetivos: Transformar SecureGuard en una plataforma de seguridad robusta con infraestructura en nube.

Funcionalidad	Descripción
Infraestructura en nube (AWS/Azure)	Despliegue de servidor de firmas con Terraform (ver FD01 §4.2.7)
Dashboard web de telemetría	Panel web para administradores (React + FastAPI)
Análisis de comportamiento avanzado	Machine learning para detección de amenazas zero-day
Soporte Linux nativo	Empaquetado <code>.deb</code> y <code>.rpm</code> para distribuciones Linux
Motor YARA integrado	Análisis con reglas YARA para detección de familias de malware
API REST para integración empresarial	Endpoints para integración con SIEM y herramientas de seguridad
Escaneo de memoria RAM	Detección de amenazas en procesos activos
VPN integrada básica	Protección adicional de red para el usuario
GitHub Pages — Landing page	Sitio web del proyecto con descarga directa
Certificación ISO/IEC 27001 (evaluación)	Evaluación de conformidad con estándares de seguridad

6. Flujo de Trabajo con Ramas (GitFlow)

6.1. Modelo de Ramas

SecureGuard Antivirus adopta el modelo **GitFlow** adaptado para equipos pequeños, con las siguientes ramas principales:



6.2. Descripción de Ramas

Rama	Propósito	Creada desde	Se fusiona a
main	Código estable en producción. Solo recibe merges de release/* o hotfix/*	—	—
develop	Rama de integración continua. Contiene el código más reciente	main	main (via release)
feature/*	Desarrollo de nuevas funcionalidades	develop	develop
fix/*	Corrección de bugs no críticos	develop	develop
release/*	Preparación de una nueva versión (freeze de funcionalidades)	develop	main + develop
hotfix/*	Correcciones críticas en producción	main	main + develop

6.3. Proceso de Trabajo Estándar

1. Crear una rama de funcionalidad:

```
git checkout develop
git pull origin develop
git checkout -b feature/nombre-funcionalidad
```

2. Desarrollar y commitear con Conventional Commits:

```
git add .
git commit -m "feat: descripción de la funcionalidad implementada"
```

3. Abrir Pull Request hacia develop: - Asignar al menos un revisor (el otro integrante del equipo) - Vincular el Issue correspondiente con **Closes #N** - El PR debe incluir descripción de cambios y capturas de pantalla si aplica

4. Revisión y merge: - El revisor aprueba o solicita cambios - Una vez aprobado, se fusiona mediante **Squash and Merge** para historial limpio

5. Eliminar la rama fusionada:

```
git branch -d feature/nombre-funcionalidad
git push origin --delete feature/nombre-funcionalidad
```

6.4. Flujo de Release

```
# Crear rama de release
git checkout -b release/v1.0 develop

# Actualizar número de versión en Cargo.toml
# Generar notas de release
# Pruebas finales de integración

# Fusionar a main y etiquetar
git checkout main
git merge --no-ff release/v1.0
git tag -a v1.0 -m "Release v1.0 - MVP SecureGuard Antivirus"
git push origin main --tags

# Fusionar de vuelta a develop
git checkout develop
git merge --no-ff release/v1.0
```

7. GitHub Projects

7.1. Estructura del Tablero

El proyecto utiliza **GitHub Projects** con un tablero de tipo Kanban organizado en las siguientes columnas:

Columna	Descripción	Criterio de entrada	Criterio de salida
Backlog	Issues definidos pero no iniciados	Issue creado y estimado	Asignado a un sprint
En Progreso	Trabajo actualmente en desarrollo	Asignado a un integrante	PR abierto
En Revisión	PR abierto esperando revisión de código	PR creado	PR aprobado
Completado	Issues cerrados y fusionados	PR mergeado	—

7.2. Etiquetas (Labels) del Proyecto

Etiqueta	Color	Descripción
feature	Verde	Nueva funcionalidad
bug	Rojo	Defecto o error en el sistema
documentation	Azul	Tarea de documentación
enhancement	Amarillo	Mejora de funcionalidad existente
security	Naranja	Relacionado con seguridad del sistema
testing	Morado	Pruebas unitarias o de integración
priority: high	Rojo oscuro	Alta prioridad
priority: medium	Amarillo	Prioridad media
priority: low	Gris	Baja prioridad

7.3. Milestones del Proyecto

Milestone	Fecha Límite	Issues Asociados	Descripción
Fase 1 — Análisis	18/04/2026	#1, #2, #3	Documentación inicial y análisis del sistema
Fase 2 — Diseño	09/05/2026	#4, #5, #6	Arquitectura y modelo de calidad
Fase 3 — Desarrollo	13/06/2026	#7-#14	Implementación de todos los módulos
Fase 4 — Pruebas	04/07/2026	#15-#18	Pruebas dinámicas y verificación
Fase 5 — Cierre	18/07/2026	#19, #20	Gestión de calidad y entrega final

7.4. Burndown y Métricas de Progreso

Se utilizan las siguientes métricas para el seguimiento del proyecto:

- **Velocity por sprint:** Número de story points completados por semana
- **Issue closure rate:** Porcentaje de issues cerrados en el tiempo planificado
- **PR review time:** Tiempo promedio desde apertura hasta merge de un Pull Request
- **Código commiteado:** Líneas de código nuevas/modificadas por semana

8. Conclusiones

1. **Gestión con GitHub:** El uso de GitHub como plataforma central permite una gestión integral del proyecto, combinando control de versiones,

seguimiento de tareas, revisión de código y documentación en un único entorno colaborativo.

2. **GitFlow adaptado:** El modelo GitFlow implementado garantiza la estabilidad del código en producción (**main**) mientras permite el desarrollo paralelo de funcionalidades en ramas separadas, reduciendo el riesgo de integración.
 3. **Roadmap estructurado:** La planificación en tres versiones (v1.0 MVP, v1.1 mejoras, v2.0 plataforma) permite un desarrollo incremental y sostenible, con objetivos claros y medibles en cada etapa.
 4. **Trazabilidad:** La vinculación de commits, Pull Requests e Issues garantiza la trazabilidad completa desde el requerimiento hasta la implementación, facilitando las auditorías académicas y el mantenimiento futuro.
 5. **Documentación viva:** El uso de GitHub Wiki como centro de documentación técnica asegura que la información esté siempre actualizada y accesible para todos los integrantes y futuros contribuyentes del proyecto.
-